

Chapitre 5 : les collections

Une **collection** en PL/SQL est un **ensemble ordonné d'éléments du même type**, identifiés par un **indice**.

◇ Les 3 types de collections

Type de collection	Caractéristiques principales
Tableaux associatifs	- Indice : entier ou chaîne - Pas de constructeur - Pas d'ordre ni taille fixe
Tables imbriquées	- Ensemble non ordonné - Indices numériques - Peut être initialisée avec un constructeur
Tableaux prédimensionnés (VARRAY)	- Taille fixée à la déclaration - Ordre conservé - Doit être initialisé avant usage

◇ Collections multidimensionnelles

Toutes les collections sont **unidimensionnelles**, mais il est **possible de simuler la multidimensionnalité** en définissant **une collection de collections** :

```
TYPE TypeLigne IS TABLE OF NUMBER;
TYPE Matrice IS TABLE OF TypeLigne;
```

→ On déclare d'abord les types, puis on crée des variables à partir de ces types.

◇ Déclaration des types

▶ Tableau associatif

```
TYPE TypeTab IS TABLE OF VARCHAR2(100)
INDEX BY BINARY_INTEGER;
```

Utilisation directe sans initialisation :

```
TabErreur(-20001) := 'Erreur détectée';
```

▶ Table imbriquée

```
TYPE TypeListe IS TABLE OF VARCHAR2(64);
TablePannes TypeListe := TypeListe('Urgente', 'Grave');
```

⚠ Non initialisée → ORA-06531

▶ Tableau prédimensionné (VARRAY)

```
TYPE TypeDep IS VARRAY(10) OF VARCHAR2(64);
TableDep TypeDep := TypeDep('IT', 'RH');
```

⚠ Dépassement de taille → ORA-06532

◇ Méthodes associées aux collections

Méthode	Description
EXISTS(n)	Teste si l'élément à l'indice n existe
COUNT	Nombre d'éléments présents
LIMIT	Taille max (uniquement pour VARRAY)
FIRST, LAST	Premier et dernier indice
NEXT(i)	Indice suivant
EXTEND	Ajoute un ou plusieurs éléments (VARRAY uniquement)
TRIM	Supprime les derniers éléments
DELETE	Supprime un ou plusieurs éléments (sauf VARRAY)

⚠ Seule EXISTS fonctionne sur une collection vide.

⚠ EXTEND, TRIM, DELETE ne s'appliquent **pas** aux tableaux associatifs.

◇ Exceptions fréquentes

Exception	Cause
NO_DATA_FOUND	Accès à un élément inexistant
VALUE_ERROR	Indice invalide ou null
COLLECTION_IS_NULL	Collection non initialisée
SUBSCRIPT_BEYOND_COUNT	Suppression d'éléments hors limites
ORA-06532	Indice trop grand pour VARRAY

Chapitre 6 :des records aux collections

1. Déclarations

- Les **records** sont des structures de données hétérogènes (comme les structures en C).
- Déclaration possible :

```
TYPE TypeDept IS RECORD (  
  NumDep VARCHAR2(6),  
  NomDep VARCHAR2(40),  
  NumSecu VARCHAR2(8)  
);  
UnDepartement TypeDept;
```

- On peut aussi utiliser %TYPE ou %ROWTYPE :

```
TYPE TypeDept IS RECORD (  
  NumDep Departements.NumDep%TYPE,  
  NomDep Departements.NomDep%TYPE,  
  NumSecu Departements.NumSecu%TYPE  
);
```

2. Initialisation et accès aux champs

- Notation pointée : record.champ

```
UnDepartement.NumDep := 99;  
DBMS_OUTPUT.PUT_LINE(UnDepartement.NomDep);
```

3. Comparaison de 2 records

- On peut tester l'égalité de deux records avec IF Rec1 = Rec2 THEN ...
- Tous les champs doivent être du même type et dans le même ordre.

4. Utilisation des records dans les opérations SQL

Recherche d'un seul tuple :

```
SELECT * INTO UnEmploye FROM Employes WHERE NumSecu = '121212';
```

- Déclenche NO_DATA_FOUND si rien n'est trouvé.
- Déclenche TOO_MANY_ROWS si plusieurs tuples sont retournés.

Recherche de plusieurs tuples :

```
SELECT * BULK COLLECT INTO LesEmployes FROM Employes WHERE NumDep = 'd11111';
```

- Pas d'exception NO_DATA_FOUND si aucun résultat.

5. FORALL et collection bulk

- FORALL exécute un lot d'instructions SQL en une seule fois.
- Syntaxe :

```
FORALL i IN LesNrEmployes.FIRST..LesNrEmployes.LAST  
DELETE FROM empTemp WHERE NumSecu = LesNrEmployes(i);
```

6. FORALL et ROLLBACK

- Si une exception non gérée se produit, toutes les modifications précédentes sont annulées.
- Si l'exception est gérée, les modifications précédentes sont conservées.

7. Gestion des exceptions dans FORALL

- Utilisation de SAVE EXCEPTIONS :
 - Permet de continuer même en cas d'erreur.
 - Erreurs disponibles dans SQL%BULK_EXCEPTIONS.

```
FORALL i IN 1..10 SAVE EXCEPTIONS  
DELETE ...
```

8. BULK collections

- Récupérer plusieurs enregistrements en une seule fois :

```
SELECT * BULK COLLECT INTO MEmployes FROM Employes;
```

- BULK COLLECT peut aussi s'utiliser avec FETCH + LIMIT.

9. RETURNING ... BULK COLLECT INTO

- Utilisé pour récupérer les valeurs modifiées dans des collections :

```
FORALL i IN ...  
DELETE FROM empTemp WHERE ...  
RETURNING NumSecu, Nom, Prenom BULK COLLECT INTO LesNum, LesNoms, LesPrenoms;
```

10. FORALL et BULK COLLECT ensemble

- Utilisation conjointe dans des procédures efficaces de traitement de masse :
 - BULK COLLECT pour charger.
 - FORALL pour exécuter une série d'opérations.

Rappels Importants (fin du cours)

- SELECT ... INTO → exception NO_DATA_FOUND si aucun résultat.
- FOR ... IN (SELECT ...) → pas d'exception si vide.
- BULK COLLECT n'entraîne **pas** d'exception NO_DATA_FOUND même si aucun résultat.
- SQL%BULK_ROWCOUNT(i) donne le nombre de lignes affectées à chaque itération.

CHAPITRE 9 – LES CURSEURS

◇ 1. Introduction

- Un **curseur** permet de **parcourir un ensemble de tuples** retournés par une requête SQL.
- Il **relie SQL (set-based)** et PL/SQL (**record-based**).
- Agit comme un **pointeur** sur les résultats d'une requête.
- La requête n'est **évaluée qu'à l'ouverture du curseur**.
- Les résultats sont **figés** jusqu'à la fermeture.

◇ Curseurs explicites (explication simplifiée)

Un **curseur explicite** permet de **parcourir manuellement** le résultat d'une requête, ligne par ligne.

Étapes d'utilisation :

1. CURSOR c IS ... → Déclaration
2. OPEN c → Ouverture (exécution de la requête)
3. FETCH c INTO ... → Lecture d'une ligne
4. CLOSE c → Fermeture du curseur

Utilisation recommandée :

- Besoin de traiter les résultats **ligne par ligne**
- Nécessité d'un **contrôle précis** du traitement
- Accès aux attributs :
%FOUND, %NOTFOUND, %ROWCOUNT

◇ Curseurs FOR implicites

Ce sont les plus simples : **tout est géré automatiquement** par PL/SQL.

Utilisation :

```
FOR var IN (SELECT * FROM table) LOOP
  -- traitement
END LOOP;
```

Avantages :

- Aucun OPEN, FETCH, CLOSE à écrire
- Pas besoin de déclaration de curseur

Limites :

- Pas d'accès aux attributs %FOUND, %ROWCOUNT, etc

◇ Curseurs FOR explicites

Mélange des deux : on **déclare le curseur explicitement**, mais **PL/SQL le gère dans une boucle FOR**.

Utilisation :

```
CURSOR c IS SELECT ...;
FOR var IN c LOOP
  ...
END LOOP;
```

Avantages :

- Plus lisible que la version manuelle
- Accès aux attributs %ROWCOUNT, %ISOPEN, etc.

◇ Curseurs avec paramètres

Permettent de **rendre un curseur dynamique** en passant des valeurs à son ouverture.

✓ Déclaration :

```
CURSOR c (param TYPE) IS SELECT ... WHERE champ = param;
```

✓ Ouverture :

```
OPEN c('valeur');
```

🔗 Utile pour :

- Réutiliser le même curseur avec différents filtres
- Rendre un traitement plus flexible

◇ Curseurs variables (REF CURSOR)

Aussi appelés curseurs **dynamiques**. Leur requête est **déterminée au moment de l'exécution**.

✓ Déclaration :

```
TYPE TypeCurseur IS REF CURSOR;  
MonCurseur TypeCurseur;
```

✓ Utilisation :

```
OPEN MonCurseur FOR SELECT ...;  
FETCH MonCurseur INTO ...;
```

🔗 Deux types :

- **Typé** (RETURN type) → + sécurisé
- **Non typé** → + flexible (toute requête possible)

◇ Curseurs FOR UPDATE

Permettent de **verrouiller les lignes** retournées par un curseur pour les modifier.

✓ Syntaxe :

```
SELECT ... FROM ... FOR UPDATE [OF colonne] [NOWAIT];
```

✓ Mise à jour de la ligne lue :

```
UPDATE table SET ... WHERE CURRENT OF curseur;
```

🔗 Utilisé pour :

- Modifier des données en sécurité dans une boucle
- Éviter les conflits d'accès concurrents

◇ Commit et les curseurs

Un COMMIT **ferme les curseurs FOR UPDATE**.

⚠ Ne jamais faire :

```
-- Mauvais :  
FOR rec IN curseur LOOP  
  UPDATE ...  
  COMMIT; -- ❌ Ferme le curseur !  
END LOOP;
```

✓ Solution recommandée :

- Soit COMMIT **après la boucle**
- Soit utiliser la **clé primaire** au lieu de CURRENT OF

◇ Le curseur implicite SQL

PL/SQL crée un curseur implicite pour chaque **instruction DML** : INSERT, UPDATE, DELETE, SELECT INTO.

✓ **Avantages :**

- Pas besoin de le déclarer
- Utilisable directement avec :
 - SQL%FOUND
 - SQL%NOTFOUND
 - SQL%ROWCOUNT

✓ **Exemple :**

```
UPDATE ...;  
IF SQL%NOTFOUND THEN  
  INSERT ...;  
END IF;
```

Chapitre 10 – Les Déclencheurs (Triggers) en PL/SQL

1. Qu'est-ce qu'un déclencheur ?

Un **trigger** est un mini-programme PL/SQL qui s'exécute **automatiquement** quand une action spécifique se produit sur une table (ou une vue).

Il agit **en coulisse**, sans que tu aies à l'appeler !

À quoi ça sert ?

-  Générer automatiquement des clés primaires
-  Empêcher des modifications non autorisées
-  Maintenir la cohérence entre plusieurs tables
-  Enregistrer des actions dans des journaux (audit)
-  Gérer les mises à jour en cascade

2. Syntaxe de base

```
CREATE OR REPLACE TRIGGER nom_trigger
BEFORE | AFTER INSERT | UPDATE | DELETE [OF colonne]
ON table
[FOR EACH ROW]
[WHEN (condition)]
BEGIN
-- code PL/SQL ici
END;
```

Astuce :

- FOR EACH ROW → déclencheur ligne à ligne
- WHEN → exécute le trigger seulement si la condition est vraie

3. Cas concrets : des déclencheurs utiles !

Empêcher une baisse de salaire

```
WHEN OLD.sal > NEW.sal → raise_application_error
```

Refuser une date de naissance dans le futur

→ Comparaison avec CURRENT_DATE

Vérifier qu'un chef appartient bien à son département

→ Contrôle avec une requête SELECT

Mises à jour en cascade (auteurs/livres)

→ Modifier ou supprimer les enregistrements liés automatiquement

Auditer toutes les modifications

→ Stocker chaque action (INSERT, DELETE, UPDATE) dans une table audit

Générer une clé automatiquement

→ Utilisation d'une **séquence** ou calcul avec LPAD, MAX, etc.

4. Gérer ses déclencheurs

- **Activer/désactiver :**

```
ALTER TRIGGER mon_trigger DISABLE;
```

```
ALTER TRIGGER mon_trigger ENABLE;
```

- **Supprimer :**

```
DROP TRIGGER mon_trigger;
```

5. L'ordre d'exécution (à retenir)

1. BEFORE STATEMENT (avant tout)
2. BEFORE ROW (avant chaque ligne)
3. Action SQL
4. AFTER ROW (après chaque ligne)
5. AFTER STATEMENT (après tout)

➡ Tu peux aussi **forcer un ordre** avec FOLLOWS

6. Tables mutantes (piège courant)

Un déclencheur ne peut **pas lire/modifier** une table **qu'il est en train de modifier**.

💡 Solution :

- **Stocker les données** dans une variable globale (PACKAGE)
- Utiliser un **compound trigger** avec 4 sections :
BEFORE STATEMENT, BEFORE EACH ROW, AFTER EACH ROW, AFTER STATEMENT

7. Déclencheurs INSTEAD OF (pour les vues !)

✅ Permettent de rediriger l'action vers les bonnes tables

```
CREATE OR REPLACE TRIGGER trig_vue  
INSTEAD OF INSERT ON ma_vue  
BEGIN  
  -- redirection vers les tables sources  
END;
```

8. Déclencheurs DDL & Événements système

-  Exécutés lors de :
 - CREATE, DROP, ALTER, etc.
 - Connexion (LOGON), déconnexion (LOGOFF)
 - Démarrage/Arrêt de la base

➡ Très utile pour **auditer** ou **surveiller** les actions système !

En résumé

Type de Trigger	Réagit à...	Niveau
LMD (Insert/Update/Delete)	Sur les données (table)	Ligne/Table
INSTEAD OF	Actions sur des vues	Vue
DDL	Création/suppression d'objet	Schéma
Système	Connexions, erreurs, etc.	Base/Schéma